LABORATORIO 2026

CSS avanzado

Cómo ocupa espacio cada elemento y cómo ubicarlo: box model, Flexbox y responsive.



Objetivos de la clase

Al terminar esta clase vas a poder:

1. **Entender el box model**

   Contenido, `padding`, `border` y `margin`, y controlar el tamaño total con `box-sizing`.

2. **Ubicar elementos con `display` y Flexbox**

   Poner elementos en fila, alinearlos y repartir el espacio entre ellos.

3. **Adaptar una página al celular**

   Con la etiqueta `viewport`, unidades relativas y una media query.

Antes de empezar

Esta clase es para ver a tu ritmo. Verificá que tenés:

1. **La tarjeta de receta de la clase anterior**

   Con `estilos.css` vinculado: lo de hoy se apoya en selectores, colores y unidades.

2. **DevTools a mano**

   F12 abre el panel. Hoy lo usamos para ver las cajas y simular un celular.

3. **Tu carpeta `practica-css/`**

   Para escribir cada ejemplo a medida que lo veas.

01 SECCIÓN 01

Box model

Todo elemento HTML es una caja.



El box model

Documentación

Cada elemento es una caja con cuatro capas, de adentro hacia afuera.



Contenido: el texto o la imagen. Mide *`width`* × *`height`*.

`padding`: espacio interno, entre el contenido y el borde. Toma el color de fondo del elemento.

`border`: el borde, con grosor, estilo y color.

`margin`: espacio externo, que separa la caja de sus vecinas. Siempre es transparente.

Padding, border y margin en código

Documentación

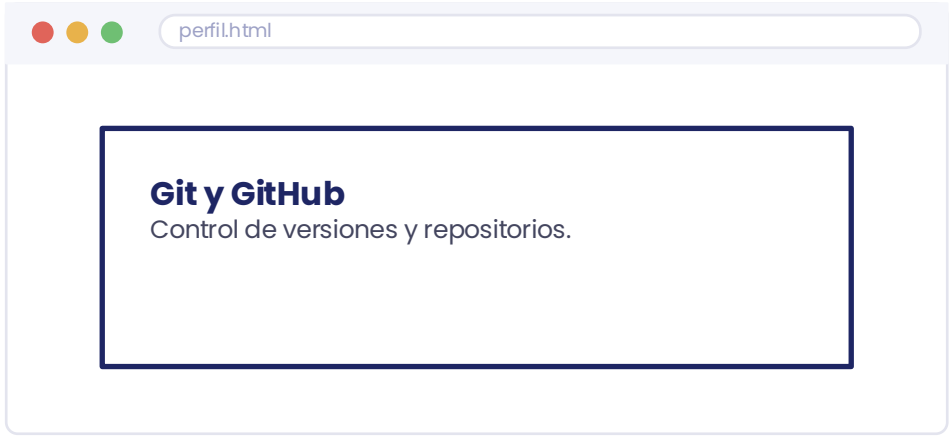
padding: **espacio interno**, entre el contenido y el borde.
border: el **borde de la caja**: grosor, estilo y color.
margin: **espacio externo**, entre la caja y sus vecinas.

Con un valor se aplica a los cuatro lados; con dos, arriba/abajo e izquierda/derecha; con cuatro: arriba, derecha, abajo, izquierda.

EJEMPLO

estilos.css

```css
.tarjeta {
  padding: 20px;
  border: 2px solid #1E2664;
  margin: 16px 0;
}
```



ERROR COMÚN

Confundir *padding* con *margin*: para separar el texto de su borde es *padding*; para separar dos cajas, *margin*.

Ver las cajas en DevTools

El navegador te dibuja el box model de cualquier elemento.

1

Inspeccioná un elemento

Clic derecho → Inspeccionar, o F12.

2

Buscá la pestaña «Calculado» (Computed)

Arriba de todo aparece el diagrama de cajas con las medidas reales de `margin`, `border`, `padding` y contenido.

3

Pasá el mouse por el diagrama

La página resalta cada capa con un color distinto: así ves dónde termina una y empieza la otra.

02 SECCIÓN 02

box-sizing

Cuánto mide de verdad una caja.



¿Cuánto mide la caja?

Documentación

width: el **ancho** de la caja.
Por defecto, *width* mide **solo el contenido**: el *padding* y el *border* se **suman por fuera**.

EJEMPLO

estilos.css

```css
.tarjeta {
  width: 300px;
  padding: 20px;
  border: 1px solid #C9CBDE;
}
```

Así se ve el ancho en pantalla



1 + 20 + 300 + 20 + 1 = 342px de ancho total

border (1) · padding (20) · width (300) · padding (20) · border (1)

Ancho real en pantalla: 300 + 20 + 20 + 1 + 1 = **342px**, no 300px.

content-box vs *border-box*

✕ content-box (por defecto)

estilos.css

```css
.tarjeta {
  width: 300px;
  padding: 20px;
}
```



Ocupa **340px**: el *padding* se suma por fuera del *width*.

✓ border-box

estilos.css

```css
* { box-sizing: border-box; }
.tarjeta {
  width: 300px;
  padding: 20px;
}
```



Ocupa **300px**: el *padding* se descuenta de adentro.
* selecciona **todos los elementos**: se escribe una vez, al principio del CSS.

Recomendación: empezá siempre tu hoja de estilos con *`* { box-sizing: border-box; }`*: así cada caja mide lo que dice su *width*.

Práctica — Box model

1. Bajá los archivos de **esta carpeta** y abrí *boxmodel.html* con Live Server.
2. *.ancho*: que la caja mida *300px* de ancho.
3. *.relleno*: separá el texto del borde con *padding*.
4. *.separada*: separá las cajas A y B con *margin*.
5. *.medida*: calculá su ancho total y comprobalo en DevTools (pestaña «Calculado»).
6. Agregale *box-sizing: border-box* y fijate cuánto mide ahora.

Nota: en la slide siguiente está el **resultado esperado**, para que compares el tuyo cuando termines.

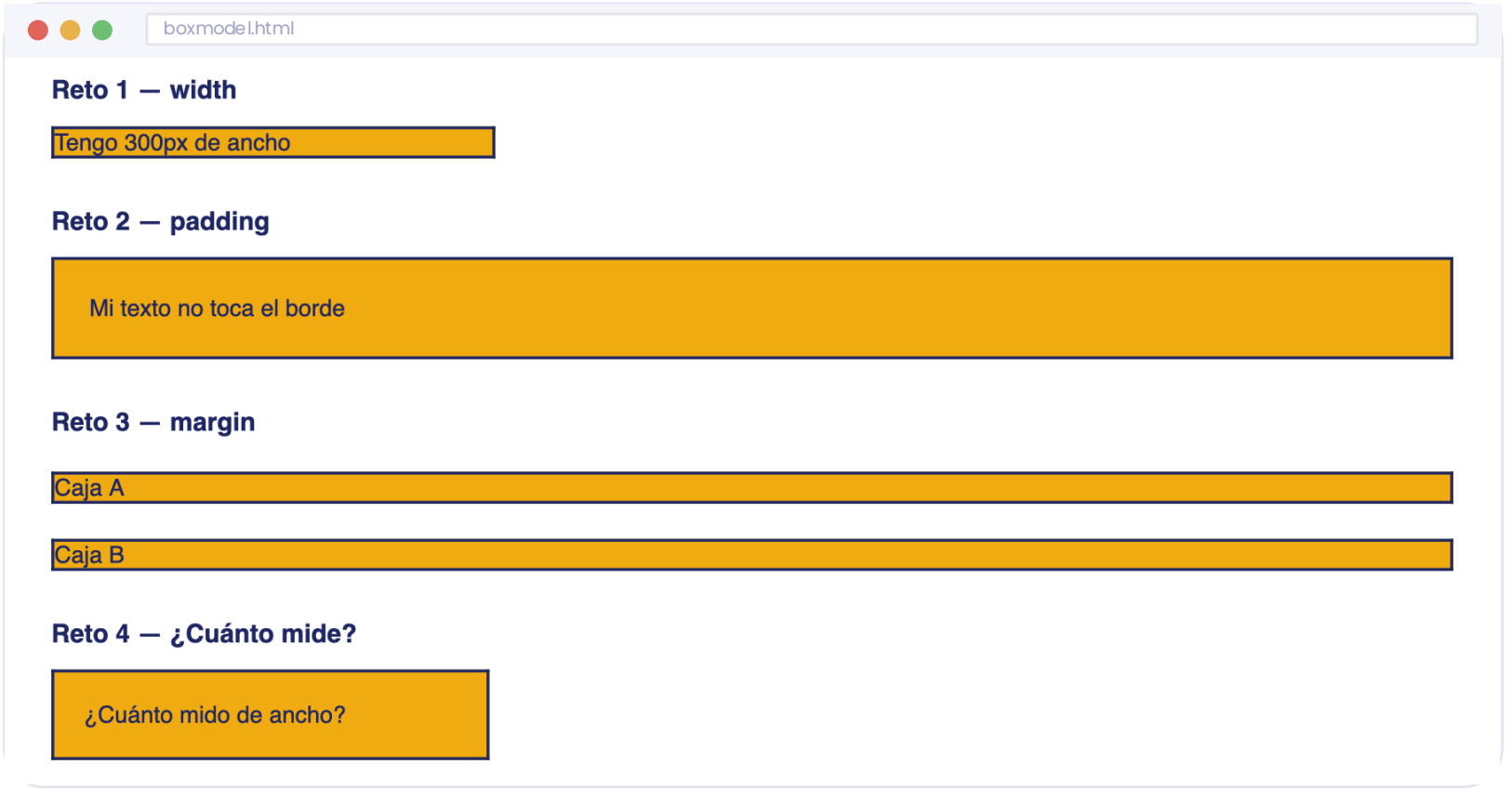
boxmodel.html

```
<div class="caja ancho">
  Tengo 300px de ancho
</div>
<div class="caja relleno">
  Mi texto no toca el borde
</div>
<div class="caja separada">Caja A</div>
<div class="caja separada">Caja B</div>
<div class="caja medida">
  ¿Cuánto mido de ancho?
</div>
```

Nota: un elemento puede tener **varias clases**, separadas por un espacio. En *class="caja ancho"*, *.caja* le da el estilo común a todas las cajas y *.ancho*, es otro estilo aplicado a este elemento especifico.

Resultado esperado

Deberían llegar a un resultado parecido a este.



03 SECCIÓN 03

display

Cómo se ubica cada caja respecto de las demás.



Valores de *display*

Documentación

1

block

Ocupa todo el ancho y empieza en una línea nueva.
Así son `<div>`, `<p>` y `<h1>`.

2

inline

Va **dentro de la línea** de texto e ignora `width` y `height`.
Así son `<a>` y `<span>`.

3

inline-block

Va en la línea, como `inline`, pero **acepta** `width`, `height` y `padding` completos.

4

none

El elemento **desaparece** de la página, sin dejar hueco.

Un enlace con forma de botón

display: con *inline-block*, el elemento sigue en la línea pero acepta *width*, *height* y *padding*.
Un *<a>* es *inline*: sin esto, no toma *width* y su *padding* no empuja a lo que tiene arriba y abajo.

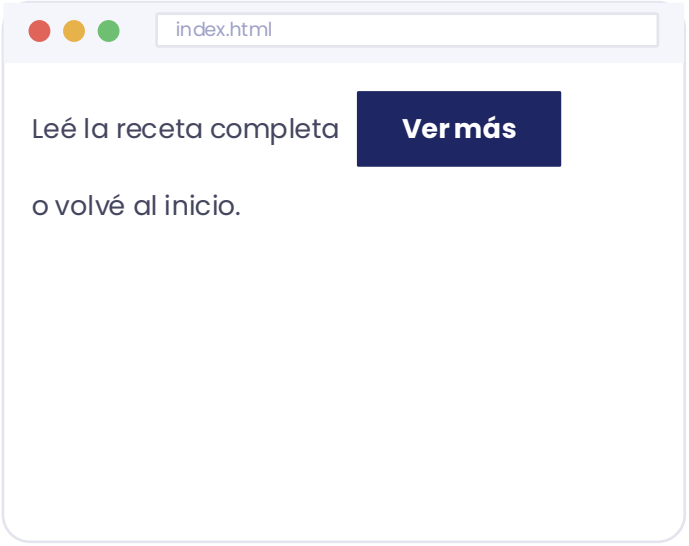
EJEMPLO

index.html

```
<p>
  Leé la receta completa
  <a href="#">Ver más</a>
  o volvé al inicio.
</p>
```

estilos.css

```
a {
  display: inline-block;
  width: 140px;
  padding: 10px;
  text-align: center;
  color: white;
  background-color: #1E2664;
}
```



ERROR COMÚN

Darle *width* a un *<a>* y no ver ningún cambio (por defecto se comporta como *inline*): le falta *display: inline-block* (o *block*).

04 SECCIÓN 04

Flexbox

Poner elementos en fila, alinearlos y repartir el espacio.



Contenedor e ítems

Documentación

display: con ***flex***, los hijos directos del **contenedor** pasan a ser ítems y se ponen en fila.
flex-direction:
row los pone en fila (por defecto);
column, uno debajo del otro.
gap: el espacio entre los ítems, sin sumarles ***margin*** uno por uno.

EJEMPLO

index.html

```html
<main class="tarjetas">
  <article class="tarjeta">1</article>
  <article class="tarjeta">2</article>
  <article class="tarjeta">3</article>
</main>
```

estilos.css

```css
.tarjetas {
  display: flex;
  flex-direction: row;
  gap: 24px;
}
```



Repartir el espacio: *justify-content*

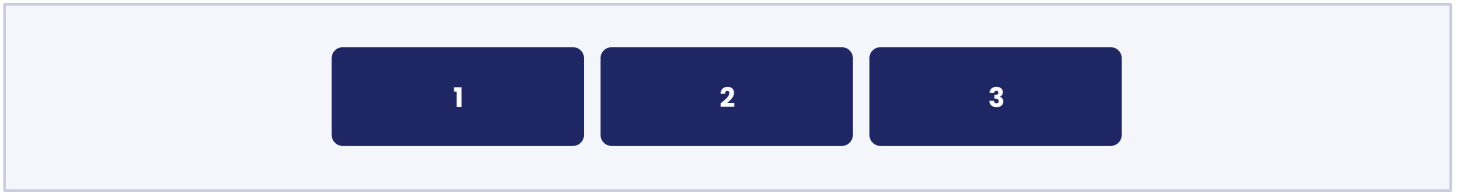
Documentación

Cómo se ubican los ítems a lo largo de la fila.

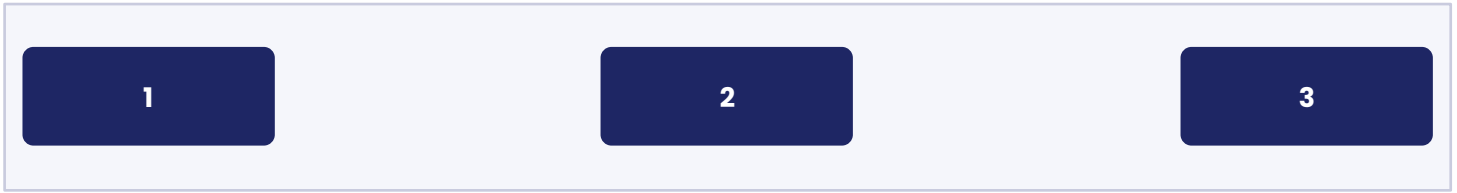
justify-content: flex-start;
Al principio (por defecto)



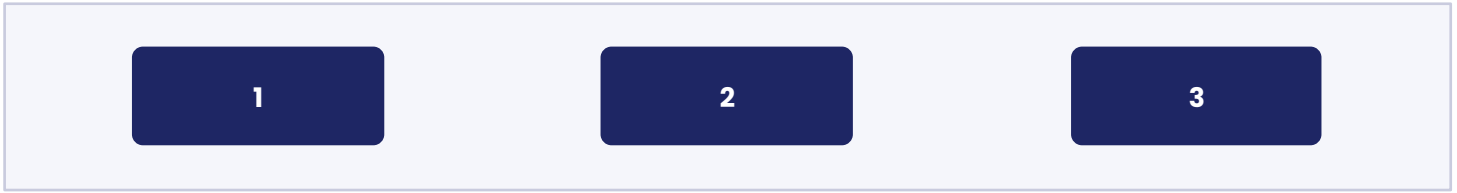
justify-content: center;
En el centro



justify-content: space-between;
Repartidos: el primero y el último, pegados a los bordes



justify-content: space-around;
Mismo espacio a cada lado: en los bordes queda la mitad



Un menú en una sola fila

Documentación

justify-content: reparte los ítems a lo largo de la fila.
align-items: los alinea en el otro eje (arriba, centro, abajo).

Juntos arman el menú clásico: logo a la izquierda y enlaces a la derecha.

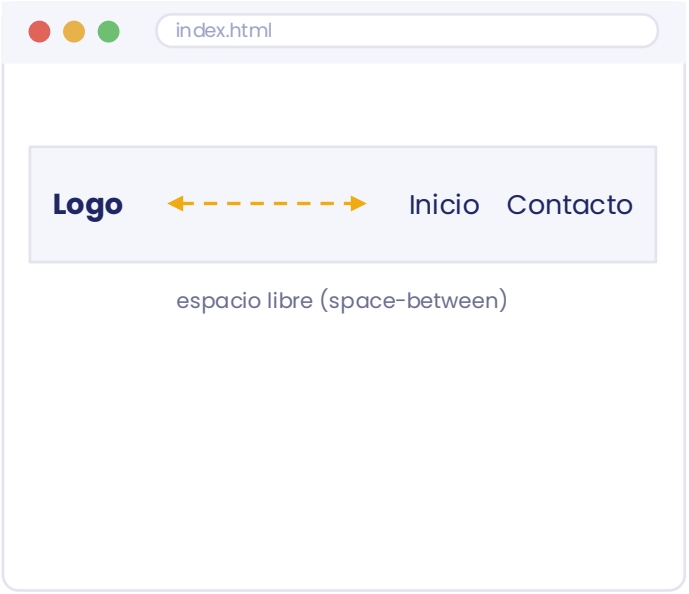
EJEMPLO

index.html

```html
<nav class="menu">
  <a class="logo">Logo</a>
  <ul class="enlaces">
    <li>Inicio</li>
    <li>Contacto</li>
  </ul>
</nav>
```

estilos.css

```css
.menu {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.enlaces {
  display: flex;
  gap: 16px;
}
```



Galería: *inline-block* vs Flexbox

✕ Con inline-block

estilos.css

```css
.tarjeta {
  display: inline-block;
  width: 30%;
  margin-right: 3%;
}
```

Hay que calcular anchos y márgenes a mano.
Los saltos de línea del HTML aparecen como huecos entre tarjetas.

✓ Con Flexbox

estilos.css

```css
.tarjetas {
  display: flex;
  gap: 24px;
}
```

El contenedor reparte el espacio entre sus hijos.
gap define la separación exacta, sin márgenes sueltos.

Cuando no entran: *flex-wrap*

Documentación

flex-wrap: con ***wrap***, los ítems que no entran bajan a la fila siguiente.
Por defecto, Flexbox mete todos los ítems en una sola fila aunque se achiquen.

EJEMPLO

index.html

```html
<main class="tarjetas">
  <article class="tarjeta">1</article>
  <article class="tarjeta">2</article>
  <article class="tarjeta">3</article>
</main>
```

estilos.css

```css
.tarjetas {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
}
.tarjeta { width: 300px; }
```



La tercera tarjeta no entra en la fila y baja sola, sin aplastar a las otras dos.

Práctica — Flexbox

1. Bajá los archivos de **esta carpeta** y abrí *flexbox.html* con Live Server.
2. *.fila*: poné las cajas en fila, separadas con *gap*.
3. *.centrado*: centralas con *justify-content*.
4. *.menu*: logo a la izquierda y enlaces a la derecha, alineados al centro.
5. *.galeria*: que las fotos que no entran bajen de fila.
6. *.columna*: poné las cajas una debajo de la otra.

Nota: en la slide siguiente está el **resultado esperado**, para que compares el tuyo cuando termines.

flexbox.html

```
<section class="fila">
  <div class="caja">1</div>
  <div class="caja">2</div>
  <div class="caja">3</div>
</section>
<nav class="menu">
  <span class="logo">Mi sitio</span>
  <ul class="enlaces">...</ul>
</nav>
<section class="galeria">
  <div class="foto">foto 1</div>
  ...
</section>
```

Resultado esperado

Deberían llegar a un resultado parecido a este.



05 SECCIÓN 05

Responsive

Que la página se vea bien en una compu y en un celular.



La etiqueta viewport

Documentación

Una página **responsive** se adapta al **ancho de la pantalla**.
El primer paso es avisarle al celular que use su ancho real, con esta etiqueta en el *<head>*.

EJEMPLO

index.html

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

VS Code ya la incluye cuando generás la estructura con *!* + Tab.

Medidas que se adaptan

Documentación

1

%

Relativo al contenedor: `width: 90%` acompaña el ancho de la pantalla.

2

max-width

Pone un tope: `max-width: 960px` evita que el contenido se estire de más en pantallas grandes.

3

rem

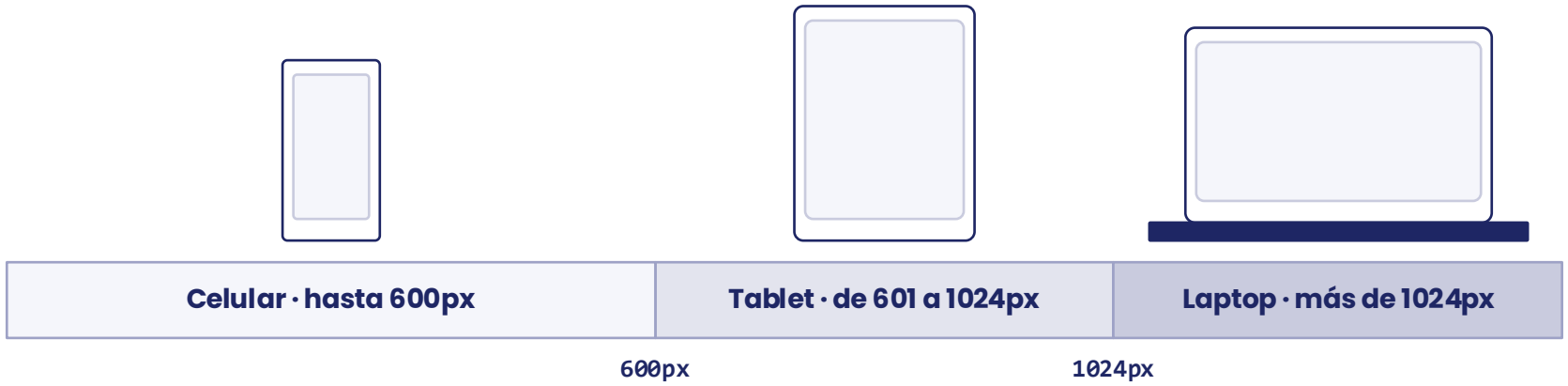
Relativo a la letra de la raíz: los textos y espaciados escalan juntos.

Media queries

Documentación

Una **media query** aplica reglas solo cuando se cumple una condición, por ejemplo un ancho máximo. Fuera de la condición, esas reglas se ignoran.

TAMAÑOS DE PANTALLA



`@media (max-width: 600px)` → sus reglas se aplican solo en **celulares**.

`@media (max-width: 1024px)` → se aplican en **tablets y celulares** (todo lo que mida 1024px o menos).

Para probarla, achicá la ventana o activá el modo dispositivo de DevTools (Ctrl+Shift+M).

Un diseño, tres pantallas

Documentación

Mismo HTML que en «Contenedor e ítems». Cada media query cambia el ancho de `.tarjeta`.

estilos.css

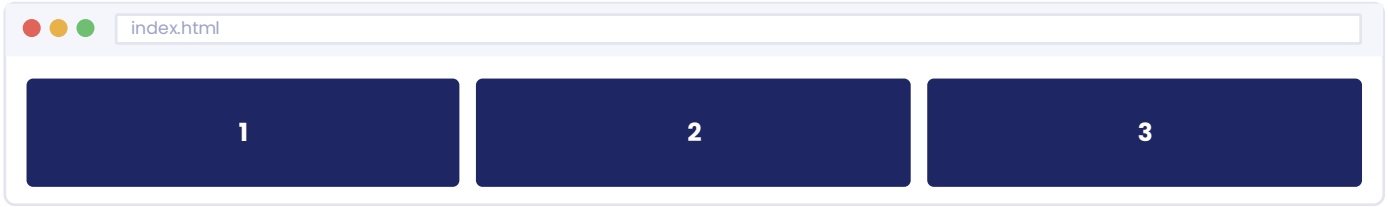
```css
.tarjetas {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
}

/* Laptop: 3 por fila */
.tarjeta { width: 30%; }

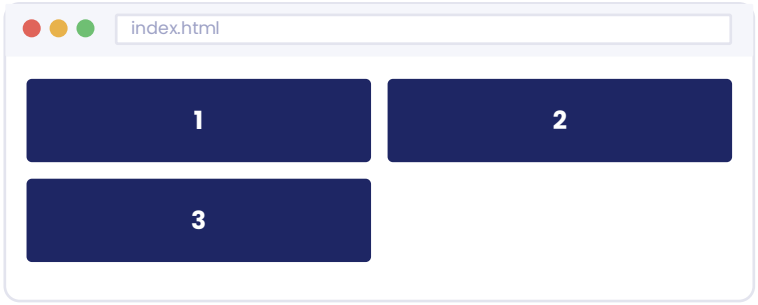
/* Tablet: 2 por fila */
@media (max-width: 1024px) {
  .tarjeta { width: 45%; }
}

/* Celular: 1 por fila */
@media (max-width: 600px) {
  .tarjeta { width: 100%; }
}
```

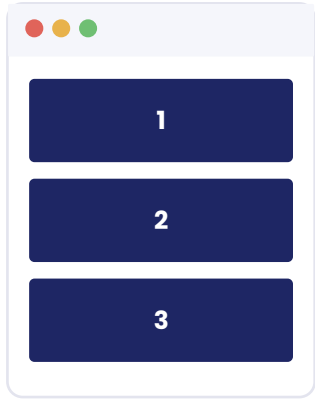
Laptop (más de 1024px) · 3 por fila



Tablet (601 a 1024px) · 2 por fila



Celular (600px o menos) · 1 por fila



Repaso: ¿Verdadero o Falso?

Respondé antes de pasar a la siguiente slide.

1. El `padding` es el espacio entre una caja y sus vecinas. V F

2. Con `box-sizing: border-box`, el `padding` y el `border` quedan dentro del `width`. V F

3. Un `<a>` con `display: inline` respeta el `width` que le pongamos. V F

4. `display: flex` se declara en el contenedor, no en cada ítem. V F

5. Las reglas de `@media (max-width: 600px)` se aplican en pantallas de 600px o menos. V F

Repaso: respuestas

La opción correcta queda marcada en dorado.

1. El `padding` es el espacio entre una caja y sus vecinas. — V **F**
2. Con `box-sizing: border-box`, el `padding` y el `border` quedan dentro del `width`. — **V** F
3. Un `<a>` con `display: inline` respeta el `width` que le pongamos. — V **F**
4. `display: flex` se declara en el contenedor, no en cada ítem. — **V** F
5. Las reglas de `@media (max-width: 600px)` se aplican en pantallas de 600px o menos. — **V** F

06 SECCIÓN 06

Para practicar...

Una página de perfil con tarjetas y menú.



Ejercicio — página de perfil

1. Bajá los archivos de **esta carpeta** y abrí *perfil.html* con Live Server.
2. Aplicá *box-sizing: border-box* a todos los elementos.
3. Dale a *.tarjeta* *padding*, *border* y *margin*.
4. Poné *.menu* en fila: logo a la izquierda y *.enlaces* a la derecha, también en fila.
5. Ubicá las tarjetas una al lado de la otra con *gap* y *flex-wrap*, repartidas con *justify-content: space-between*.
6. Con una media query, apilalas en pantallas de menos de 600px.

Nota: en la slide siguiente está el **resultado esperado**, para que compares el tuyo cuando termines.

perfil.html

```
<nav class="menu">
  <a href="#" class="logo">Franco Spinelli</a>
  <ul class="enlaces">...</ul>
</nav>
<main id="proyectos" class="tarjetas">
  <article class="tarjeta">
    <h2>Git y GitHub</h2>
  </article>
  ...
</main>
```

Resultado esperado

Deberían llegar a un resultado parecido a este.





¿Cómo sé si quedó bien?

1

Cada tarjeta tiene aire adentro y separación afuera

En DevTools, el diagrama de cajas muestra su *`padding`* y su *`margin`*.

2

El menú ocupa una sola fila

Logo a la izquierda, enlaces a la derecha y en fila.

3

Las tarjetas van una al lado de la otra

Y bajan solas a otra fila cuando no entran.

4

En el celular, las tarjetas se apilan

Probalo con el modo dispositivo de DevTools (Ctrl+Shift+M).



¿Preguntas?